

UNIT- IV

ARM Programming using C: Simple C Programs using Function Calls – Pointers – Structures - Integer and Floating Point Arithmetic - Assembly Code using Instruction Scheduling – Register Allocation - Conditional Execution and Loops.

ARM Programming using C:

C Compilers and Optimization:

‘C’ compilers have to translate C function literally into assembler so that it works for all possible inputs. In practice, many of the input combinations are not possible.

```
Let us consider the example,  
Void memclr(Char *data, int N)  
{  
  For(;N>0;N-1)  
  {  
    *data = 0;  
    data++;  
  }  
}
```

The function of this example is (memclr) it clears N bytes of memory at address “data”.

To write efficient C code, you must be aware of areas where the C compiler has to be conservative, the limits of the processor architecture the C compiler is mapping to and the limits of specific C compiler.

The C compiler must be conservative and assume all possible value for N and all possible alignment for data.

Optimizing code takes time and reduce source code readability.

Basic C Data Type:

ARM Processor have 32 bit registers and 32 bit processing operations there are no arithmetic and logical instructions to manipulate values in memory directly, so u must to load values from memory to registers before processing them,

Early versions of ARM Architecture (ARM v1, ARM v3) provided hardware support for loading and storing unsigned 8-bit and unsigned/signed 32-bit values.

Compilers armcc and gcc use the data types given in the following table:

C Data Type	Implementation
Char	Unsigned 8-bit byte
Short	Signed 16-bit half word
Int	Signed 32-bit word
Long	Signed 32-bit

	word
Long long	Signed 64-bit doubleword

The exceptional case for typechar is worth nothing as it can cause problems when porting the code from another processor architecture.

Local Variable Types:

- The ARM v4 processor can efficiently load and store 8, 16, and 32 bit data but most of ARM data processing operations use 32 bit data only.
- For this reason 32-bit data type int or long is used as local variable wherever possible.
- Avoid using char and short as local variable type even If the operation is a 8-bit or 16-bit value.
- Eg: Using a char type variable i as a loop counter with loop continuation condition $i \geq 0$.
- As i is unsigned for ARM compilers the loop will never terminate.
- Armcc compiler produces warning in this situation, unsigned comparison with 0.
- Compilers also provide an override switch to make char signed.
- For this example the command line option – fsigned – char will make char signed on gcc
- The command line option – ZC will have same effect with armcc.

Function Arguments:

Converting local variables from types char or short to type int increases performance and reduce code size. The same holds for function arguments. Consider the following simple function, which adds two 16-bit values, having the second and returns a 16-bit sum,

```
Short add_vi(short a, short b)
{
    Return a+(b>>1);
}
```

This function is a little artificial, but it is a useful test case to illustrate the problems faced by the compiler. The input values a, b and the return value will be passed in 32-bit ARM registers. Should the compiler assume that these 32-bit values are in the range of a short type, that is, -32,768 to +32,767? Or should the compiler force values to the in this range by sign-extending the lowest 16-bits to fill the 32-bit register? The compiler must make compatible decisions for the function caller and callee. Either the caller or callee must perform the cast to a short type.

We say that function arguments are passed wide if they are not reduced to the range of the type and narrow if they are.

Signed Versus Unsigned Types:

- If your code uses addition, subtraction and multiplication, then there is no performance difference between signed and unsigned operations.
- However difference arises when it comes to division.

Efficient use of C Types for ARM Processors:

Following are some of C types for ARM Processors,

- 1) For local variables held in registers, don't use char or short type unless 8-bit or 16-bit modular arithmetic is necessary.
- 2) Use signed or unsigned int type instead unsigned types are faster when you use decisions.

- 3) For array entries and global variables held in main memory, use the type with the smallest size possible to hold the require data. This saves memory footprint. The ARM v4 Architecture is efficient at loading and storing all data widths provided you traverse array by incrementing the array pointer. Avoid using offset from the base of the array with short type arrays, as LDRH does not support this.
- 4) Use explicit casts when reading array entries or global variables into local variables, or writing local variables out to array entries. The cast makes it clear that for fast operation you are taking a narrow width type stored in memory and expanding it to a wider type in the registers. Switch on implicit narrowing cast warning in the compiler to detect implicit cast.
- 5) Avoid implicit or explicit narrowing cast in expression because they usually cost extra cycles. Cast on load or store are usually free because the load or store instruction itself performs the cast.
- 6) Avoid char and short types for function arguments or return values. Instead use the int type even if the range of the parameter is smaller. This prevents the compiler performing unnecessary cast.

FUNCTION CALLS

The ARM-Thumb Procedure Call Standard (ATPCS) is the one which tells about how to pass function arguments and return values in ARM registers. The recently introduced ARM-Thumb Procedure Call Standard (ATPCS) covers both the ARM and Thumb interworking more efficiently.

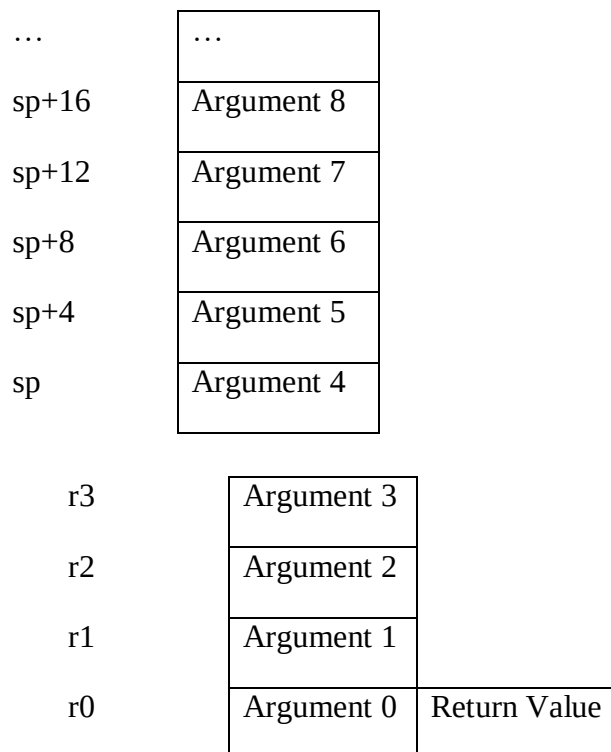


Fig: ATPCS Argument passing

- The first four integer arguments are placed in the first four ARM registers r0, r1, r2 and r3.
- The subsequent integer arguments are placed on the full descending stack ascending in memory.
- The function register integer values are placed in r0.

- For a two-word argument like long or double, a pair of consecutive argument -registers are returned in registers r0, r1.
- Now, according to the command line compiler options the compiler may pass structures in registers.

In ARM PC5, it is easier to call functions with four or fewer arguments than functions with five or more arguments. Because for functions with four or fewer arguments, the compiler can pass all the arguments in the registers. Whereas for functions with more arguments, both the caller and callee must access the stack for some arguments.

If a C function needs more than four arguments, then it is beneficial to use structures in this context, first the group related arguments must be passed and then the structure pointer was passed rather than passing the multiple arguments, because the arguments that are passed will depend on the structure of the software.

Calling function efficiently for ARM Processor:

- 1) Try to restrict functions to four arguments. This will make them more efficient to call. Use structures to group related arguments and pass the structure pointer instead of multiple arguments.
- 2) Define the small functions in the same source file and before the functions that call them.
- 3) Critical functions can be inlined using the inline keyword.

POINTER ALIASING:

- Pointers are a powerful part of C Language.
- If the address of the variable is taken, the compiler must assume that the variable can be change by any assignment through a pointer or by any function call, making it impossible to put into a register.
- This is also true for global variables, as they might have their address taken in some other function.
- This problem is known as pointer aliasing, because the pointer is known as an alias of the variable it points to.

Eg: The following function increments two timer values by a step amount.

```
Void timers1(int *t1, int *t2, int *step)
{
    *t1 += *step;
    *t2 += *step;
```

```
}
```

This compiles to,

```
Timers1 LDR r3, [r0] ; r3 = *t1
        LDR r12,[r2];r12=*step
        ADD r3, r3, r12 ; r3+=r12
        STR r3, [r0]; *t1= r3
        LDR r0, [r1]; r0=*t2
        LDR r2, [r2]; r2=*step
        ADD r0, r2; r0 += r2
        STR r0,[r1];*t2=r0
        MOV pc, r14; return
```

Here, we would expect that `*step` to be pulled from memory once and used twice. That doesn't happen. Because one rewritten to cache `*step` in a local variable, the redundant load is eliminated.

The same problem arises if we use structure accesses instead of direct pointer access. The following code also compiles inefficiently,

```
typedef struct {int step;} state;
typedef struct {int t1, t2;} timers;
void timers2 (state*state, timers*timers)
{
    timers-> t1 +=state -> step;
    timers ->t2 +=state -> step;
}
```

The compiler evaluates `state->step` twice in case `state -> step` and `timers ->t1` are at the same memory address. The solution is to create a new local variable to hold the value of `state->state` so that the compiler performs only a single load.

Avoiding Pointer Aliasing:

- Do not rely on the compiler to eliminate common sub expressions involving memory access. Instead create new local variables to hold the expression.
- This ensures the expression is evaluated only once.
- Avoid taking the address of local variables. The variable may be inefficient to access from then on.

STRUCTURE ARRANGEMENT:

The way we lay out a frequently used structure can have a significant impact on its performance and code density. There are two issues related to structures on the ARM, alignment of the structure entries and the overall size of the structure.

The ARM's load and store instructions are only guaranteed to load and store values with address aligned to the size of the access width. For this reason, ARM compilers will automatically align the start

address of a structure to a multiple of the largest access width used within the structure (usually four or eight bytes) and align entire structures to their access width by inserting padding.

For instance consider the structure,

```
struct {
    char a;
    int b;
    char c;
    short d;
}
```

The layout of the memory system is as shown

Address	+3	+2	+1	+0
+0	pad	pad	pad	a
+4	b[31,24]	b[23,16]	b[15,8]	b[7,0]
+8	d[15,8]	d[7,0]	pad	c

Fig: Layout of Memory System

To improve the memory usage, let us rearrange the elements of structure,

```
struct {
    char a;
    char c;
    short d;
    int b;
}
```

This reduces the structure size from 12 bytes to 8 bytes, with the following new layout is shown

Address	+3	+2	+1	+0
+0	d[15,8]	d[7,0]	c	a
+4	b[31,24]	b[23,16]	b[15,8]	b[7,0]

Fig: Layout of memory System with Reduced Size

However, it is a good idea to group structure elements of the same size, so that the structure layout doesn't contain unnecessary padding. The armcc compiler provides a keyword `_packed` that removes all padding. For example, the structure is,

```
packed structure {
    char a;
    int b;
    char c;
    short d;
}
```

The new memory layout-co-processor.

Address	+3	+2	+1	+0
+0	d[23,16]	d[15,8]	b[7,0]	a
+4	b[15,8]	b[7,0]	c	b[31,24]

Fig: New Memory Layout Co-processor

However, packed structures are slow and inefficient to access. The compiler emulates unaligned load and store operations by using several aligned accesses with data operations to merge the results. Only use the `_packed` keyword where space is far more important than speed and you can't reduce padding by rearrangement.

Efficient Structure Arrangement for ARM Processor:

To make the efficient structure arrangement for ARM processor we should follow these points,

- 1) Lay structures out in order to increasing element size. Start the structure with the smallest element and finish with largest.
- 2) Avoid very large structure. Instead use a hierarchy of smaller structure.
- 3) For portability, manually add padding (that would appear simplicity) into API structures so that the layout of the structure does not depend on the compiler.
- 4) Because of using enum types of API structures, the size of an enum type is compiler dependent.

FLOATING-POINT ARITHMETIC

The ARM core does not contain any actual floating-point hardware. Instead there are three options for an application which needs floating-point support. They are,

- 1) **Floating-Point Accelerator (FPA) Hardware Co-processor:** This implements a floating-point instruction set using a number of ARM co-processor instructions. However, this does not require the FPA hardware to exist within the system as a co-processor.
- 2) **Floating-Point Emulator(FPE):** FPE emulates in software the instructions that the FPA executes. This means that there is no need to recompile code for systems with or without the FPE.
- 3) **Floating-Point Library (FP Lib):** Floating-point operations are compiled into function calls to library routines rather than floating-point instructions. Although this is slower than using a FPE, it is typically two or three times faster than using the FPE. The overall code size of the system is also smaller because only the required library routines are included, rather than whole of the FPE. Therefore, the floating-point library is the route that ARM recommends for use in embedded systems and is the default one.

The recommended compiler options give the best results in terms of performance and code size. However, when writing floating-point code, keep the following things in mind,

- 1) **Floating-Point Division is Slow:** Division is typically twice as slow as addition or multiplication. Rewrite divisions by a constant into a multiplication with the inverse. For Example, $x = x/3.0$ becomes $x = d*(1.0/3.0)$ and the constant is calculated during compilation.
- 2) **Use Floats Instead of Doubles:** Float Variables consume less memory and fewer registers and are more efficient because of their lower precision. Use floats whenever their precision is good enough.
- 3) **Avoid using Transcendental Functions:** Transcendental functions like sin, exp and log are implemented using series of multiplications and additions. As a result, these operations are at least ten times slower than a normal multiply.
- 4) **Simply Floating-Point Expressions:** The compiler cannot apply many optimizations which are performed on integers to floating-point values. For example, $3*(x/3)$ cannot be optimized to x , since floating-point operations generally lead to loss of precision, Even the order of evaluation is important because $(a+b)+c$ is not the same as $a+(b+c)$. Therefore, it is beneficial to perform floating-point optimizations manually if it is known they are correct.

However, it is still possible that the floating performance will not reach the required level for a particular application. In such a case, the best approach may be to change from using floating-point to fixed-point arithmetic. When the range of values needed is sufficiently small, fixed-point arithmetic is more accurate and much faster than the floating-point arithmetic.

FIXED-POINT ARITHMETIC

ARM is an integer processor, all floating point operations must be simulated using integer arithmetic. Using fixed-point arithmetic instead of floating-point will considerably increase the performance of many operations.

Principles of Fixed-Point Arithmetic

In computing arithmetic. Fractional quantities can be approximated by using a pair of integers(n, e), the mantissa and the exponent. This pair represents the fraction, $n2^{-e}$. The exponent e can be considered as the number of digits to move n before placing the binary point.

Mantissa (n)	Exponent(e)	Binary	Decimal
01100100	-1	011001000	200
01100100	0	01100100	100
01100100	1	0110010.0	50
01100100	2	011001.00	25
01100100	3	01100.100	12.5
01100100	7	0.1100100	0.78125

Table: Principles of Fixed-Point Arithmetic

In the above table, if e is a variable quantity held in a register and unknown at compile time, then (n, e) is said to be floating-point number. If e is known in advance at compile time, then (n, e) is said to be fixed-point arithmetic number. Fixed-point numbers can be stored in standard integer variables by storing the mantissa. For fixed-point numbers, the exponent e is usually denoted by the letter q .

WRITING AND OPTIMIZING ARM ASSEMBLY CODE

Embedded software projects contain a few key subroutines that dominate system performance. By optimizing these routines we can reduce the system's power consumption and also the clock speed required for real-time operation. Optimization can turn an infeasible system into a feasible one, or an uncompetitive system into a competitive one.

Writing assembly code by hand gives us direct control of three optimization tools that we cannot use explicitly by writing C source. They are,

- 1. Instruction scheduling:** reordering the instructions in a code sequence to avoid processor stalls. Since ARM implementations are pipelined, the timing of an instruction can be affected by neighboring instructions.
 - 2. Register Allocation:** deciding how variables should be allocated to ARM registers or stack locations for maximum performance. Our goal is to minimize the number of memory access.
 - 3. Conditional Execution:** Accessing the full range of ARM condition codes and conditional instructions.
- ARM assembly will always give better performance compared to thumb assembly when a 32-bit bus is available.
 - Thumb is most useful for reducing the compiled size of C code that is not critical to performance and for efficient execution on a 16-bit data bus.

INSTRUCTION SCHEDULING

The time taken to execute instructions depends on the implementation pipeline. The following rules summarize the cycle timings for common instruction classes on the ARM9TDMI

If the condition is not met, then the instructions that are conditional on the value of the ARM condition codes in the CPSR takes one cycle.

If the condition is met, then the following rules apply,

1. ALU operations such as addition, subtraction and logical operations take one cycle. This includes a shift by an immediate value. If you use a register-specified shift, then add one cycle. If the instruction writes to the pc, then add two cycles.
2. Load instructions that load N32-bit words of memory such as LDR and LDM take N cycles to issue, but the result of the last word loaded is not available on the following cycle. The updated load address is available on the next cycle. This assumes zero-wait stage memory for an uncached system, or a cache hit for a cached system. An LDM of a single value is exceptional, taking two cycles. Of the instruction loads pc, then add two cycles.
3. Load instructions that load 16-bit or 8-bit data such as LDRB, LDRSB, LDRH, LDRSH take one cycle to issue. The load result is not available on the following two cycles. The updated load address is available on the next cycle. This assumes zero-wait state memory for an uncached system, or a cache hit for a cached system.
4. Branch instructions take three cycles.
5. Store instructions that store N values take N cycles. This assumes zero-wait-state memory for an uncached system, or a cache hit or a write buffer with N free entries for a cached system. An STM of a single value is exceptional, taking two cycles.
6. Multiply instructions take a varying number of cycles depending on the value of the second operand in the product.

To understand how to schedule code efficiently on the ARM, we need to understand the ARM pipeline and dependencies.

The ARM9TDMI processor performs five operations in parallel. They are,

1. **Fetch:** Fetch from memory the instruction at address pc. The instruction is loaded into the core and then processes down the core pipeline.
2. **Decode:** Decode the instructions that was fetched in the previous cycle. The processor also reads the input operands from the register bank, if they are not available via one of the forwarding paths.
3. **ALU:** Executes the instruction that was decoded in the previous cycle. Note this instruction was originally fetched from address pc-8 (ARM state) or pc-4 (Thumb state). Normally this involves calculating the answer for a data processing operation, or the address for a load, store, or branch operation. Some instructions may spend several cycles in this stage. For example, multiply and register –controlled shift operations take several ALU cycles.
4. **LS1:** Load or store the data specified by a load or store instruction. If the instruction is not a load or store, then this stage has no effect.
5. **LS2:** Extract and zero-or sign-extend the data loaded by a byte or halfword load instruction. If the instruction is not a load of an 8-bit byte or 16-bit halfword item, then this stage has no effect.

	pc	pc-4	pc-8	pc-12	pc-16
Instruction Address Action	Fetch	Decode	ALU	LS1	LS2

Fig: ARM9TDMI pipeline executing ARM state

The five-stage ARM9TDMI pipeline is as shown in figure. After the instruction has completed the five stages of the pipeline, the core writes the result to the register file. Here, pc points to the address of the instruction being fetched. The ALU was executing the instruction that was originally fetched from address PC-8 in parallel with fetching the instruction at address PC. If the instruction requires the result of a previous instructions that are not available, then the processor stalls. This is called pipeline hazard or pipeline interlock.

Example: This example shows the case where there is no interlock,

```
ADD r0, r0, r1
ADD r0, r0, r2
```

This instruction pair takes two cycles. The ALU calculates $r0+r1$ in one cycle. Therefore this result is available for the ALU to calculate $r0+r2$ in the second cycle.

Example: This example shows a one-cycle interlock caused by load use,

```
LDR r1, [r2, #4]
ADD r0, r0, r1
```

This instruction pair takes three cycles. The ALU calculates the address $r2+4$ in the first cycle while decoding the ADD instruction in parallel. However, the ADD cannot proceed on the second cycle because the load instruction has not yet loaded the value of r1. Therefore, the pipeline stalls for one cycle while the load instruction completes the LS1 stage. Now that r1 is ready, the processor executes the ADD in the ALU on the third cycle.

Pipeline	Fetch	Decode	ALU	LS1	LS2
Cycle 1		ADD	LDR		
Cycle 2			ADD	LDR	
Cycle 3			ADD		LDR

Fig: One-cycle interlock caused by load use

The above figure illustrates how this interlock affects the pipeline. The processor stalls the ADD instruction for one cycle in the ALU stage of the pipeline while the load instruction completes the LS1 stage. Since the LDR instruction proceeds down the pipeline, but the ADD instruction is stalled, a gap opens up between them. This gap is sometimes called a pipeline bubble. The bubble is marked with a dash symbol.

SCHEDULING OF LOAD INSTRUCTIONS

In compiled code load instruction occur frequently, accounting for approximately one-third of all instructions. So, care must be taken while scheduling the load instructions so that the pipeline stalls that don't occur can improve the performance. The compiler attempts to schedule the code as best it can, but the aliasing problems of C will limit the available optimizations. So that it cannot move a load instruction before a store instruction unless it is certain that the two pointers used do not point to the same address.

However, there are two ways in which we can alter the structure of the algorithm to avoid the cycles by using assembly. They are,

1. **Load scheduling by Preloading:** in this method of load scheduling, the data that is required is loaded at the end of the previous loop, rather than at the beginning of the current loop. In order to get the better performance improvement with little increase in code size, we don't unroll the loop. The ARM architecture is particularly well suited to this type of preloading because instructions can be executed conditionally.

Example: This assembly applies the preload method to the str_tolower function.

```

out      RN  0      ; Pointer to output string

in       RN  1      ; Pointer to input string

c        RN  2      ; Character loaded

t        RN  3      ; Scratch register

; void str_tolower_preload(char *out, char *in)
Str_tolower_preload
LRD c, [in, #1] ; c = *(in++)

loop
        SUB      t, c, #' A'      ; t=c-'A'
        CMP      t, #'Z'-'A'      ; if (t <= 'Z' - 'A')
        ADDLS    c, c, #'a'-'A'    ; c += 'a' - 'A'
        STRB     c, [out], #1      ; *(out++) = (char)c;
        TEQ      c, #0            ; test if c==0
        LDRNEB   c, [in], #1      ; if (c!=0) {c = *in++; goto

```

```

                                loop;}
BNE      loop      ;
MOV      pc, 1r     ; return

```

The schedule version is one instruction longer than the C version, but we save two cycles for each inner loop iteration. This reduces the loop from 11 cycles per character to 9 cycles per character on an ARM9TDMI.

- 2. Load Scheduling by Unrolling:** This method of load scheduling works by unrolling and then interleaving the body of the loop. For example, we can perform loop iterations i , $i + 1$, $i + 2$ interleaved. When the result of an operation from loop i is not ready, we can perform an operation from loop $i+1$ that avoids waiting for the loop i result.

Example: The assembly applies load scheduling by unrolling to the `str_tolower` function,

```

out      RN  0      ; Pointer to output string
in       RN  1      ; Pointer to input string
ca0      RN  2      ; Character 0
t        RN  3      ; Scratch register
ca1      RN  12     ; character 1
ca2      RN  14     ; character 2

; void str_tolower_unload(char *out, char *in)
Str_tolower_unload
STMFD    spt, {1r}          ; function entry
Loop_next3
LDRB     ca0, [in] #1        ; ca0 = *in++;
LDRB     ca1, [in] #1        ; ca1 = *in++;
LDRB     ca2, [in] #1        ; ca2 = *in++;
SUB      t, ca0, # 'A'        ; convert ca0 to lower case
CMP      t, 'Z', - 'A'
ADDLS    ca0, ca0, # 'a' - 'A'
SUB      t, ca1, # 'A'        ; convert ca1 to lower case
CMP      t, 'Z', - 'A'
ADDLS    ca1, ca1, # 'a' - 'A'
SUB      t, ca2, # 'A'        ; convert ca2 to lower case

```

CMP	t, # 'Z', - 'A'	
ADDLS	ca2, ca2, # 'a' - 'A'	
STRB	ca0, [out] # 1'	: *out++ = ca0;
TEQ	ca0, #0	; if (ca0!=0)
STRNEB	ca1, [out] # 1'	; *out++ = ca1;
TEQNE	ca1, #0	; if (ca0! = 0 && ca1 !=0)
STRNEB	ca2, [out] # 1'	; *out ++ = ca2;
TEQNE	Ca2, #0	; if (ca0!=0 && cal !=0 && ca2!=0)
BNE	loop_next3	; goto loop_next3;
LDMFD	Spl, {pc}	; return;

This loop is the most efficient implementation we have looked at so far. The implementation requires seven cycles per character on ARM9TDMI.

REGISTER ALLOCATION:

- In order to hold general-purpose data we can use 14 out of the 16 visible ARM registers.
- The two registers that were not used are r13 and r 15.
- For a function to be ATPCS compliant it must preserve the callee values of registers r14 to r11.
- The ATPCs also specifies that the stack should be 8-byte aligned.
- Therefore, this alignment has to be preserved by calling of subroutines.

Allocating Variables to Register Numbers

While writing an assembly routine, it is better to start by using names for the variables, rather than explicit register numbers. Because it allows us to change the allocation of variables to register number when their use doesn't overlap. Register names increase the clarity and readability of optimized code.

Mostly ARM operations are orthogonal with respect to register number i.e., specific register numbers do not have specific roles. Even though we swap all occurrences of two registers Ra and Rb in a routine, the function of the routine does not change.

However, there are several cases where the physical number of the register is important they are,

1. **Argument Registers:** The ATPCS convention defines that the first four arguments to a function are placed in registers r0 to r3. Further arguments are placed on the stack. The return value must be placed in r0.

2. **Registers used in a Load (or) Store Multiple:** Load and store multiple instructions LDM and STM operate on a list of registers in order of ascending register number. If r0 and r1 appear in the register list, then the processor will always load or store r0 using a lower address than r1 and so on.
3. **Load and store Double word:** The LDRD and STRD instructions introduced in ARMvSE operate on a pair of registers with sequential register numbers, Rd and Rd + 1. Furthermore, Rd must be an even register number.

There are several possible ways we can proceed when we run out of registers,

1. Reduce the number of registers we require by performing fewer operations in each loop. In this case we could load four words in each load multiple rather than eight.
2. Use the stack to store the last-used values to free up more registers. In this case we could store the loop counter N on the stack.
3. Alter the code implementation to free up more registers. This is the solution we consider in the following text.

Using More than 14 Local Variables:

If we require more than 14 local 32-bit variables in a routine, then we must store some variables on the stack. The standard procedure is to work outwards from the inner most loop of the algorithm, since the innermost loop has the greatest performance impact.

Examples

This examples shows how we can use the ARM assembler directives MAP (alias ^) and FIELD (alias #) to define and allocate space for variables and arrays on the processor stack. The directives perform a similar function to the struct operator in C.

```

MAP      0      ; Map symbols to offsets starting at offset 0
a        FIELD  4      ; a is 4 byte integer (at offset 0)
b        FIELD  2      ; b is 2 byte integer (at offset 4)
c        FIELD  2      ; c is 2 byte integer (at offset 6)
d        FIELD  64     ; d is an array of 64 characters (at offset 8)
length   FIELD  0      ; Length records the current offset reached
example
```

```

STMFD    spl,    {r4-r11, ; save callee registers
1n}
SUB       sp, sp, # length ; create stack frame
...
STR       r0, [sp, #a]     ; a = r0;
LDRSH     r1, [sp, #b]     ; r1 = b;
ADD       r2, sp, #d       ; r2 = &d[0]
...
ADD       sp, sp, #length  ; Restore the stack pointer
LDMFD     spl, {r4-r11, pc} ; Return
```

Making the Most of Available Registers

On load-store architecture such as the ARM, it is more efficient to access values held in registers than values held in memory. There are several tricks we can use to fit several sub-32-bit length variables into a single 32-bit register and thus can reduce code size and increase performance. Suppose we want to step through an array by a programmable increment. A common example is to step through a sound sample at various rates to produce different pitched notes.

We can express this in C code as,

```
Same = table[index];
```

```
Index += increment;
```

Commonly index and increment are small enough to be held as 16-bit values. We can pack these two variables into a single 32-bit variable `indinc`;

`Indinc = (index<<16) + increment =`

Bit 31	16	15	0
Index			

The C code translates into assembly code using a single register to hold `indinc`;

```
LDR    sample,    [table,    indinc,    LSR# 16] ; table[index]
ADD    indinc,    indinc,    indinc,    LSL # 16 ; index+=increment
```

If `index` and `increment` are 16-bit values, then putting `index` in the top 16-bits of `indinc` correctly implements 16-bit-wrap-around. In other words, `Index = (short) (index + increment)`. This can be useful if you are using a buffer where you want to wrap from the end back to the beginning (often known as circular buffer).

CONDITIONAL EXECUTION

The processor core can conditionally execute most ARM instructions. This conditional execution is based on one of 15 condition codes. If a condition is not specified, then the assembler defaults to the execute always condition (AL) and the other 14 conditions will split into seven pairs of complements. These conditions will depend on the four condition code flags N, Z, C, V stored in the `cpsr` register.

By default, ARM instructions do not update the N, Z, C, V flags in the ARM `cpsr`. For most instructions, to update these flags we append an S suffix to the instruction mnemonic. Exceptions to this are comparison instructions that do not write to a destination register. Their sole Purpose is to update the flags and so they don't require the S suffix.

By combining conditional execution and conditional setting of the flags, you can implement simple if statements without any need for branches. This improves efficiency since branches can take many cycles and also reduces code size.

The following C code converts an unsigned integer $0 \leq i \leq 15$ to a hexadecimal character c,

```
If (i<10)
{
    c = i + '0';
}
else
{
    c = i + 'A'-10;
}
```

We can write in assembly using conditional execution rather than conditional branches,

```
CMP        i, #10
ADDLO      c, i, #'0'
ADDHS      c, i, #'A'-10
```

The sequence works since the first ADD does not change the condition codes. The second ADD is still conditional on the result of the compare.

Conditional execution is even more powerful for cascading conditions.

Example: Consider the following code that detects if c is a letter.

```
If((c>='A' && c<='Z') || (c>='a' && c<='z'))
{
    letter++;
}
```

To implement this efficiently, we can use an addition or subtraction to move each range to the form $0 \leq c \leq \text{limit}$. Then we use unsigned comparisons to detect this range and conditional comparisons to chain together ranges.

The following assembly implements this efficiently,

```
SUB        temp,    c, #'A'
CMP        temp,    #'Z' - 'A'
SUBHI      temp,    c, #'a'
CMPHI      temp,    #'Z' - 'a'
ADDLS      Letter,  Letter #1
```

For more complicated decisions involving switches.

The logical operations AND and OR are related by the standard logical relations. You can invert logical expressions involving OR to get an expression involving AND, which can often be useful in simplifying or rearranging logical expressions.

LOOPING CONSTRUCTIONS

Most routines critical to performance will contain a loop. In this sections, we describe how to implement these loops efficiently in assembly and also discuss examples on how to unroll loops for maximum performance.

To constrict the loops in efficiently for ARM processor by the following conditions,

1. Use loops that count down to zero. Then the compiler does not need to allocate a register to hold the termination value and the comparison with zero is free.
2. Use unsigned loop counters by default and the continuation condition $i \neq 0$ rather than $i > 0$. This will ensure that the loop overhead is only two instructions.
3. Use do-while loops rather than for loops when you know the loop will iterate at least once. This saves the compiler checking to see if the loop count is zero.
4. Use do-while loops to reduce the loop overhead. Do not overunroll. If the loop overhead is small as a proportion of the total, then unrolling will increase code size and hurt the performance of the cache.
5. Try to arrange that the number of elements in arrays are multiples of four or eight. You can then unroll loops easily by two, four, or eight times without worrying about the leftover array elements.

Decrementing Counted Loops

For a decrementing loop of N iterations, the loop counter i counts down from N to 1 inclusive. The loop terminates with $i = 0$. An efficient implementations is,

```
MOV i, N
Loop
    ; loop body goes here and i=N, N-1, ...,1
    SUBS i, i, #1
    BGT loop
```

The loop overhead consists of a subtraction setting the condition codes followed by conditional branch. On ARM7 and ARM9 this overhead costs four cycles per loop. If i is an array index, then we count down from N-1 to 0 inclusive so that we can access array element zero.

We can implement his is the same way by using a different conditional branch

```
SUBS i, N, #1
Loop
    ; loop body goes here and i=N-1, N-2, ...,0
    SUBS i, i, #1
    BGE loop
```

In this arrangement the Z flag is set on the last iteration of the loop and cleared for other iterations. If there is anything different about the last loop, then we can achieve this using the EQ and NE conditions.

There is no reason why we must decrement by one on each loop. Suppose we require $N/3$ loops. Rather than attempting to divide N by three, it is far more efficient to subtract three from the loop counter on each iteration,

```
MOV i, N
```

```
Loop
```

```
    ; loop body goes here and iterates (round up) ( $N/3$ ) times
```

```
    SUBS i, i, #3
```

```
    BGT loop
```

Unrolled Counted Loops

Loop unrolling reduces the loop overhead by executing the loop body multiple times. Consider the C library function as a case study. This function sets N bytes of memory at address s to the byte value C . the function needs to be efficient, so we will look at how to unroll the loop without placing extra restrictions on the input operands. Our version of `memset` will have the following C prototype,

```
Void my_memset(char *s, int c, unsigned in N);
```

To be efficient for large N , we need to write multiple bytes at a time using STR or STM instructions. Therefore our first task is to align the array pointer s . However, it is only beneficial if N is sufficiently large. We aren't sure yet what "sufficiently large" means, but let's assume we can choose a threshold value T_1 and only bother to align the array when $N \geq T_1$. Clearly $T_1 \geq 3$ as there is no point in aligning if we don't have four bytes to write.

Now suppose we have aligned the array S . we can use store multiple to set memory efficiently. For example we can use a loop of four store multiples of eight word each to set 128 bytes on each loop. However it will only be worth doing this if $N \geq T_2 \geq 128$, where T_2 is another threshold to be determined later on.

Finally we are left with $N \leq T_2$ bytes to set. We can write bytes in blocks of four using STR until $N < 4$. Then we can finish writing bytes singly with STRB to the end of array.

Multiple Nested Loops

How many loop counters does it take to maintain multiple nested loops? Actually, one will suffice-or more accurately, and provided the sum of the bits needed for each loop count does not exceed 32. We can combine the loop counts within a single register, placing the innermost loop count at the highest bit positions. This section gives an example showing how to do this. We will ensure the loops count down from max -1 to 0 inclusive so that loop terminates by producing a negative result.

Other Counted Loops

Here, the value of a loop counter is as an input for calculations in the loop. It is not always desirable to count down from N to 1 or N-1 to 0. For example, to select bit out of a data register on at a time may require a power-of-two mask that doubles on each iteration.

The following subsections show useful looping structures that count in different patterns. They use only a single instruction combined with a branch to implement the loop.

Negative Indexing

This loop structure counts from $-N$ to 0 (inclusive or exclusive) in steps of size STEP.

```
RSB i, N, #0; i=-N
```

Loop

```
; loop body goes here and i=-N, -N+step, ...,
```

```
ADDS i, i, #STEP
```

```
BLT loop ; use BLT ore BLE to exclude 0 or not
```

Logarithmic Indexing

This loop structure counts done from 2^N to 1 in powers of two. For example, if $N = 4$, then it counts 16,8,4,2,1

```
MOV i, #1
```

```
MOV i, i, LSL n
```

Loop

```
; loop body
```

```
MOVS i, i, LSR#1
```

```
BNE loop
```

The following loop structure counts down from an N-bit mast to a one-bit mask. For example, if $N=4$, then it counts 15,7,3,1.

```
MOV i, #1
```

```
RSB i, i, i, LSL N ; i=(1<<N)-1
```

Loop

```
; loop body
```

```
MOVS i, i, LSR#1
```

```
BNE loop
```

2 MARKS

1. What is meant by alias?

Two pointers are said to *alias* when they point to the same address. If you write to one pointer, it will affect the value you read from the other pointer. In a function, the compiler often doesn't know which pointers can alias and which pointers can't. The compiler must be very pessimistic and assume that any write to a pointer may affect the value read from any other pointer, which can significantly reduce code efficiency.

2. Compare ARMCC and GCC?

The *ARMCC* in *ADS1.1* will treat *Bool* as a one-byte type as it only uses the values 0 and 1. *Bool* will only take up 8 bits of space in a structure. However, *GCC* will treat *Bool* as a word and take up 32 bits of space in a structure. To avoid ambiguity it is best to avoid using enum types in structures used in the API to your code.

3. Define LS1 and LS2?

LS1: Load or store the data specified by a load or store instruction. If the instruction is not a load or store, then this stage has no effect.

LS2: Extract and zero- or sign-extend the data loaded by a byte or half word load instruction. If the instruction is not a load of an 8-bit byte or 16-bit half word item, then this stage has no effect

4. Define preloading and unrolling?

Preloading: In this method of load scheduling, we load the data required for the loop at the end of the previous loop, rather than at the beginning of the current loop. To get performance improvement with little increase in code size, we don't unroll the loop.

Unrolling: This method of load scheduling works by unrolling and then interleaving the body of the loop. For example, we can perform loop iterations i , $i + 1$, $i + 2$ interleaved. When the result of an operation from loop i is not ready, we can perform an operation from loop $i + 1$ that avoids waiting for the loop i result.

5. Define pointer?/ What is the role of pointers? (May 2017)

Pointer is a memory variable used to store the memory address of another variable.

6. Why we need to allocate registers?

You can use 14 of the 16 visible ARM registers to hold general-purpose data. The other two registers are the stack pointer *r13* and the program counter *r15*. For a function to be ATPCS compliant it must preserve the callee values of registers *r4* to *r11*. ATPCS also specifies that the stack should be eight-byte aligned; therefore you must preserve this alignment if calling subroutines

7. What is the need of local variables?

If you need more than 14 local 32-bit variables in a routine, then you must store some variables on the stack. The standard procedure is to work outwards from the innermost loop of the algorithm, since the innermost loop has the greatest performance impact.

8. What is unrolled counter loop?

Loop unrolling reduces the loop overhead by executing the loop body multiple times. Loops are a common construct in most programs. Because a significant amount of execution time is often spent in loops, it is worthwhile paying attention to time-critical loops. Small loops can be unrolled for higher

performance, with the disadvantage of increased code size. When a loop is unrolled, a loop counter needs to be updated less often and fewer branches are executed. If the loop iterates only a few times, it can be fully unrolled so that the loop overhead completely disappears. The compiler unrolls loops automatically at -O3 -O time. Otherwise, any unrolling must be done in source code.

9. Why multiple nested loops are used?

Actually, one will suffice—or more accurately, one provided the sum of the bits needed for each loop count does not exceed 32. We can combine the loop counts within a single register, placing the inner most loop count at the highest bit positions. This section gives an example showing how to do this. We will ensure the loops count down from $max - 1$ to 0 inclusive so that the loop terminates by producing a negative result.

10. What is full descending stack?

A **Descending** stack grows downwards. It starts from a high memory address, and as items are pushed onto it, progresses to lower memory addresses. The first four integer arguments are passed in the first four ARM registers: *r0*, *r1*, *r2*, and *r3*. Subsequent integer arguments are placed on the full descending stack.

11. What is fetch and decode instruction?

Fetch: Fetch from memory the instruction at address *pc*. The instruction is loaded into the core and then processes down the core pipeline.

Decode: Decode the instruction that was fetched in the previous cycle. The processor also reads the input operands from the register bank if they are not available via one of the forwarding paths.

12. Define the rules for generate a structure. (May 2017)

The following rules generate a structure with the elements packed for maximum efficiency :

- Place all 8-bit elements at the start of the structure.
- Place all 16-bit elements next, then 32-bit, then 64-bit.
- Place all arrays and larger elements at the end of the structure.
- If the structure is too big for a single instruction to access all the elements, then group the elements into substructures.

The compiler can maintain pointers to the individual substructures.

13. What is meant by ATPCS? (ARM-thumb procedure call standard)

The *ARM-Thumb Procedure Call Standard* (ATPCS) ensures that separately compiled or assembled subroutines can work together. It describes how to pass function arguments and return values in ARM registers.

14. What is the need of the register such as R0, R1, R2, R3?

These registers are called as the argument register. These holds the first four function arguments of function call and return value on function return. A function may corrupt these registers and use them general scratch registers within the function.

15. How to call functions efficiently for ARM?

- 1) Try to restrict functions to four arguments. This will make them more efficient to call. Use structures to group related arguments and pass the structure pointer instead of multiple arguments.
- 2) Define the small functions in the same source file and before the functions that call them.
- 3) Critical functions can be inlined using the inline keyword.

16. How to create a structure in ARM programming using C? (Nov 2016)

To make the efficient structure arrangement for ARM processor we should follow these points,

- 1) Lay structures out in order to increasing element size. Start the structure with the smallest element and finish with largest.
- 2) Avoid very large structure. Instead use a hierarchy of smaller structure.
- 3) For portability, manually add padding (that would appear simplicity) into API structures so that the layout of the structure does not depend on the compiler.
- 4) Because of using enum types of API structures, the size of an enum type is compiler dependent.

17. Mention any two uses of register allocation. / Tell the purpose of register allocation (May 2016) (Nov 2016)

- General purpose registers of ARM are used to hold data.
- It preserves the callee values of the registers
- It preserves the alignment of calling subroutines

18. What is FPE?

Floating-Point Emulator(FPE): FPE emulates in software the instructions that the FPA executes. This means that there is no need to recompile code for systems with or without the FPE.

19. List out the rules to follow while writing floating point code.

- Floating point division is slow
- Use floats instead of doubles
- Avoid using transcendental functions
- Simplify floating point expressions

20. What is code optimization?

Code optimization is any method of code modification to improve code quality and efficiency. A program may be optimized so that it becomes a smaller size, consumes less memory, executes more rapidly, or performs fewer input/output operations.

21. What are the optimization tools used for writing assembly code?

- Instruction scheduling
- Register allocation
- Conditional execution

22. What are the ways in which the structure of algorithm can be altered to avoid pipeline stalls?

- Load scheduling by preloading
- Load scheduling by unrolling

23. Write an 8051 C program to send values 00-FF to port P1 (May 2016)

```
#include<reg51.h>
void main(void)
{
    unsigned char z;
    for(z=0;z<=255;z++)
        P1=z;
}
```

24. When loop is used in a program? (Nov 2017)

1. Use loops that count down to zero. Then the compiler does not need to allocate a register to hold the termination value and the comparison with zero is free.
2. Use unsigned loop counters by default and the continuation condition $i \neq 0$ rather than $i > 0$. This will ensure that the loop overhead is only two instructions.
3. Use do-while loops rather than for loops when you know the loop will iterate at least once. This saves the compiler checking to see if the loop count is zero.
4. Use do-while loops to reduce the loop overhead. Do not overunroll. If the loop overhead is small as a proportion of the total, then unrolling will increase code size and hurt the performance of the cache.
5. Try to arrange that the number of elements in arrays are multiples of four or eight. You can then unroll loops easily by two, four, or eight times without worrying about the leftover array elements.

25. Write a simple ARM program using function calls. (Nov 2017)

```
#include <stdio.h>

extern int myadd(int a, int b);

int main()
{
    int a = 4;
    int b = 5;
    printf("Adding %d and %d results in %d\n", a, b, myadd(a, b));
    return (0);
}
```